Run Cache

Related links:

- Fivetran Technical Documentation here
- Instructions on how to set up in Prod

`dbt run cache` is our way of executing model runs, driven by the desire to speed up development and reduce cost. The associated slack channel is *#prj_analytics_dbt_run_cache*

Features

1. For **unselected** models, defers parent relationship to prod models
2. For **selected** models, skips execution or clones if a version already exists
3. If optimizations are not possible, should work exactly like standard dbt

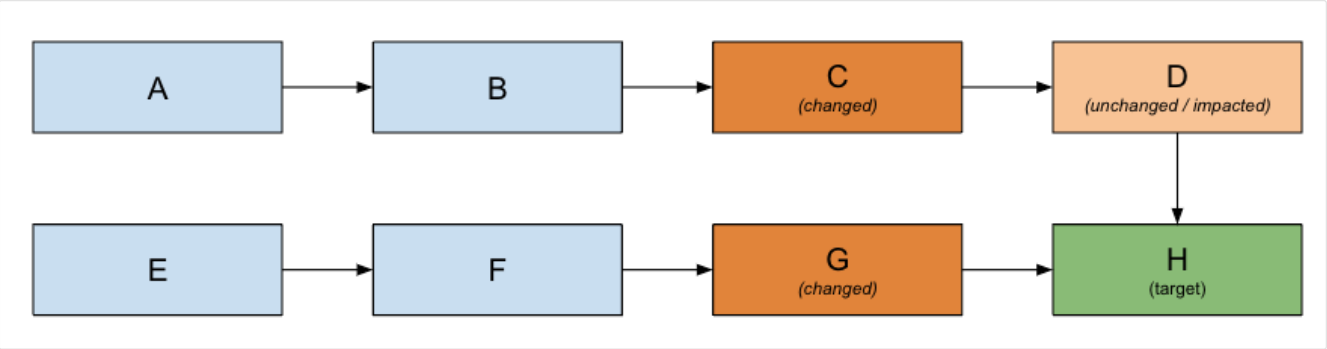
Getting Started - Initial Setup

- Step 1: Run `gmorn` — this will automatically be installed in your virtual environment.
- Step 2: Add this prod target to your dbt profile located in `~/.dbt/profiles.yml`
- Step 3: Execute `dbt run -s dummy` to test that you're set up
  - 3a: On first run, you will need to authenticate. A screen will open in your browser, asking for your email address. You will be authenticated through SSO.

Usage

Run cache works with native dbt commands. You can use vanilla dbt and benefit from cache, or you can use enhanced selectors to benefit even further. Consider the following two scenarios, which should cover majority of your run needs.

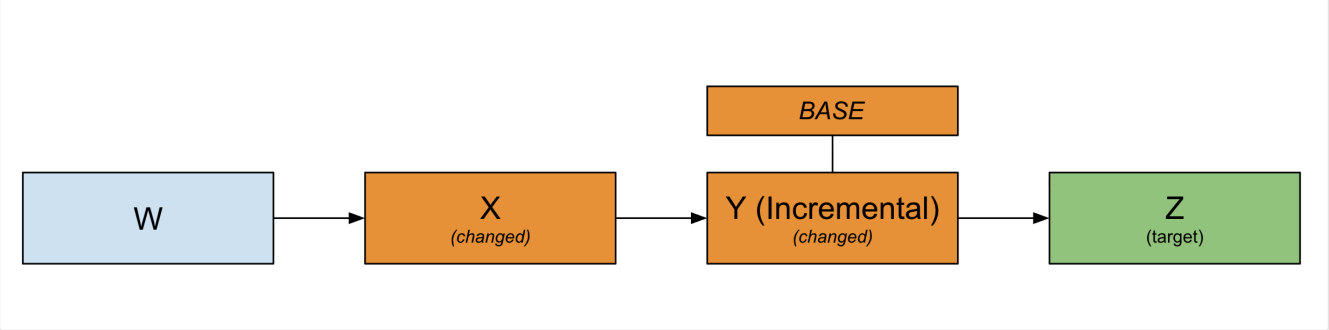
DAG with only Tables/Views (no Incremental)



Command	Result
dbt run -s 'git:main+,+H' --favor-state	Mimics existing smart run behavior

	• Runs target model (H) and changed models (C, D, G) • defers to **prod** for parent models (F & B) **note, this is def of the --favor-state flag
dbt run -s H	• Runs target model (H) • defers to **local schema** for parent models (D & G)
dbt run -s +H	• Runs all models upstream of target (A, B, C, D, E, F, G) • Runs target model (H) • If Run Cache is enabled in Prod, **A, B, E, F** will be CLONED

DAG, with an incremental model



Command	Result
dbt run -s 'git:main+,+Z' --favor-state	• defers to W in prod • runs X • clone BASE from prod. Then run incremental Y • if BASE not in prod, then run incremental Y on top of dev • run Z
dbt run -s 'git:main+,+Z' --favor-state --full-refresh	• defers to W in prod runs X

	• full refresh Y • run Z
DBT_RUN_CACHE_CLONE_INCREMENTAL_IN_DEV=NEVER dbt run -s 'git:main+,+Z' --favor-state	• defers to W in prod • runs X • Run incremental Y on top of dev • run Z

Other Commands for Reference

Command	Result
dbt run -s 'git:main+,+target_model' --favor-state	<MOST COMMON> Mimics Smart Run, where: • Only one target • Target has been modified
dbt run -s 'git:main+,+target_model_a' -s 'git:main+,+target_model_b' --favor-state	Smart Run, where: • Multiple targets • Targets have been modified
dbt run -s 'git:main' --favor-state	Run all changed models
dbt run -s 'git:main+' --favor-state	Run all changed models AND everything after

Advanced Usage & FAQs:

Disabling dbt run cache

dbt run cache can be disabled for a specific dbt command invocation by using the `DBT_RUN_CACHE_DISABLED` environment variable. Ex:

- `DBT_RUN_CACHE_DISABLED=1 dbt …`

You can disable it for your current terminal session by doing `export DBT_RUN_CACHE_DISABLED=1` and then running your dbt commands like normal. This can be undone by doing `unset DBT_RUN_CACHE_DISABLED`. Ex:

- `export DBT_RUN_CACHE_DISABLED=1`
- `dbt run …`
- `dbt run …`
- *<want to re-enable dbt run cache>*
- `unset DBT_RUN_CACHE_DISABLED`

Flag options

`--favor-state`

- **Impacts?** This flag impacts only **deferred models** (models that are not in your selector but are the parents of models in your selector).
- **What is does:** When this flag is added, deferrals will point to prod models, if they're available. If the prod models are not available, deferrals will point to your local models — i.e., `coalesce(prod_model, local_schema_model)`
- Note: the default behavior for run cache is to point all deferred models to your local schema models. If the local models are not available, deferrals will point to prod models — i.e., `coalesce(local_schema_model, prod_model)`

Enabled Team-Level Configurations

Note - All of these team-level configurations can be customized to your needs.

Freshness

- This is a concept used to ensure that a model's data is up-to-date relative to its upstream sources and the passage of time, and to control when models should be rebuilt.
- Given our pipelines run so frequently and our transformations run regularly throughout the day, we utilize the following optional configuration so that we can leverage the SKIP/NO-OP functionality in run cache.
  - `run_cache_freshness_tolerance` - this time configuration indicates our tolerance window for stale data.
- Helps answer the question: "Is the data recent enough?"

Non-deterministic tolerances

- This concept exist to handle the reality that some models do not produce identical results every time they run, even when the inputs and logic haven't changed.
  - For example, some of our models use `NOW()` or `current_timestamp()` resulting in runs never yielding the same results.
- `run_cache_tolerate_nondeterminism` enables us the ability to indicate that we are okay with the differences that would result from the expected fluctuations the non-deterministic functions produce.
- Helps answer the question: "Is the change significant enough?"

Clone Incremental in Dev

- By default, incremental models will auto clone from prod incremental tables if the table does not already exist in your development environment.

- This feature is useful because when you run an incremental model in prod it doesn't full refresh the table but rather add the new logic on top of what already exist in production. This config allows user to better mimic what their change will look like in production.
- `run_cache_clone_incremental_in_dev` allows us to override this functionality.
  - Projects are configured to instead always clone from prod on each dbt invocation (`ALWAYS`).
  - Another option is to disable this functionality entirely so that clones are not automatically created (`NEVER`)

How these are configured

These configurations are set at the team level, in our `dbt_project.yml` file. If you want to override these variables, there are two ways of doing so.

1. Update your profiles.yml file. Example:

```
fivetran:
  target: dev
  outputs:
    dev:
      type: bigquery
      project: digital-arbor-400
      dataset: z_dbt_kristin
      method: oauth
      priority: interactive
      retries: 1
      threads: 32
      timeout_seconds: 360000
      run_cache_freshness_tolerance: 5 days (or the threshold you want)
      run_cache_tolerate_nondeterminism: true (or false)
      run_cache_clone_incremental_in_dev: always (or if_table_missing, never)
```

2. Add environment variables to the beginning of your individual dbt runs.

```
DBT_RUN_CACHE_FRESHNESS_TOLERANCE="1 hour" dbt run -s model_name

DBT_RUN_CACHE_TOLERATE_NONDETERMINISM=1 dbt run -s model_name

DBT_RUN_CACHE_CLONE_INCREMENTAL_IN_DEV="always" dbt run -s model_name
```

Troubleshooting Q&A

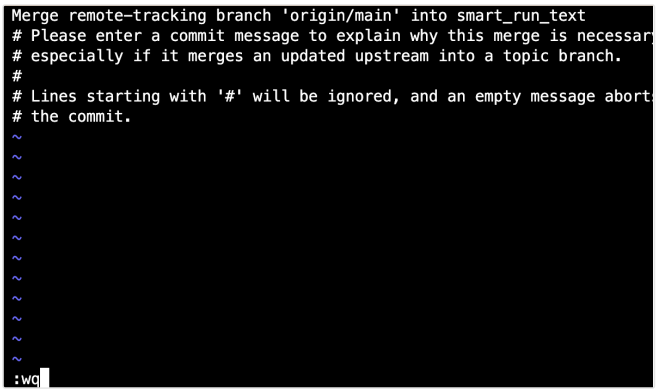
Q: When I run `dbt run -s 'git:main+,+target_model' --favor-state` it runs a significantly higher number of models than I anticipated.

A: Often this is due to either:

a. the model you are changing affecting more models then you would think
   - our DAG is quite complicated — are you isolating your code change to one area of the DAG, or are you touching models that have many in-between models?

b. Your local branch being out of date // not up-to-date with the latest version of main.
   - You will need to take the following steps:

- Checkout origin/main (your local machine's version of main)
  - `$ git checkout main`
- Pull changes from main to your local
  - `$ git pull`
- Checkout the branch you're working on
  - `$ git checkout working_branch_name`
- Merge origin/main to your working branch
  - `$ git merge origin/main`
- From here, one of two things will happen:
  - No merge conflicts: VIM will open, prompting you to write a commit message. When this happens, type in `:wq` to write & quit program. Git's standard commit message will be saved.



  - Merge conflicts: resolve merge conflicts. You will need to do this prior to merging the PR anyway :)